

INTRODUCTION AU LANGAGE



Moussa LO
M2ITIC/IFOAD 2014/2015
Université Gaston Berger de Saint-Louis

Bibliographie Java (1/2)

- **Le langage Java**, K.Arnlod, J. Gosling, Thomson, 1996.
- **Java La synthèse – Des concepts objet aux architectures Web**, G. Clavel, et al., Dunod, 2000.
- **Introduction à la programmation objet en Java**, J. Brondeau, Dunod, 1999.
- **Le langage Java – Programmer par l'exemple**, T. Leduc, D. Leduc, Technip, 2000.
- **Algorithmique et programmation objet en Java**, V. Granet, Dunod, 2001.

Bibliographie Java (2/2)

- **Initiation à l'algorithmique objet**, A. Cardon, C. Dabancourt, Eyrolles, 2001.
- **Thinking in Java**, B. Eckel, Prentice Hall, 2000.
Disponible sous forme électronique à l'URL
<http://www.BruceEckel.com>
- **Le poly de Java**, H. Garetta, Polycopié, Université de la Méditerranée, 2003.
- Site Java de Sun : <http://java.sun.com>



INTERFACES GRAPHIQUES

Interfaces graphiques : AWT & Swing

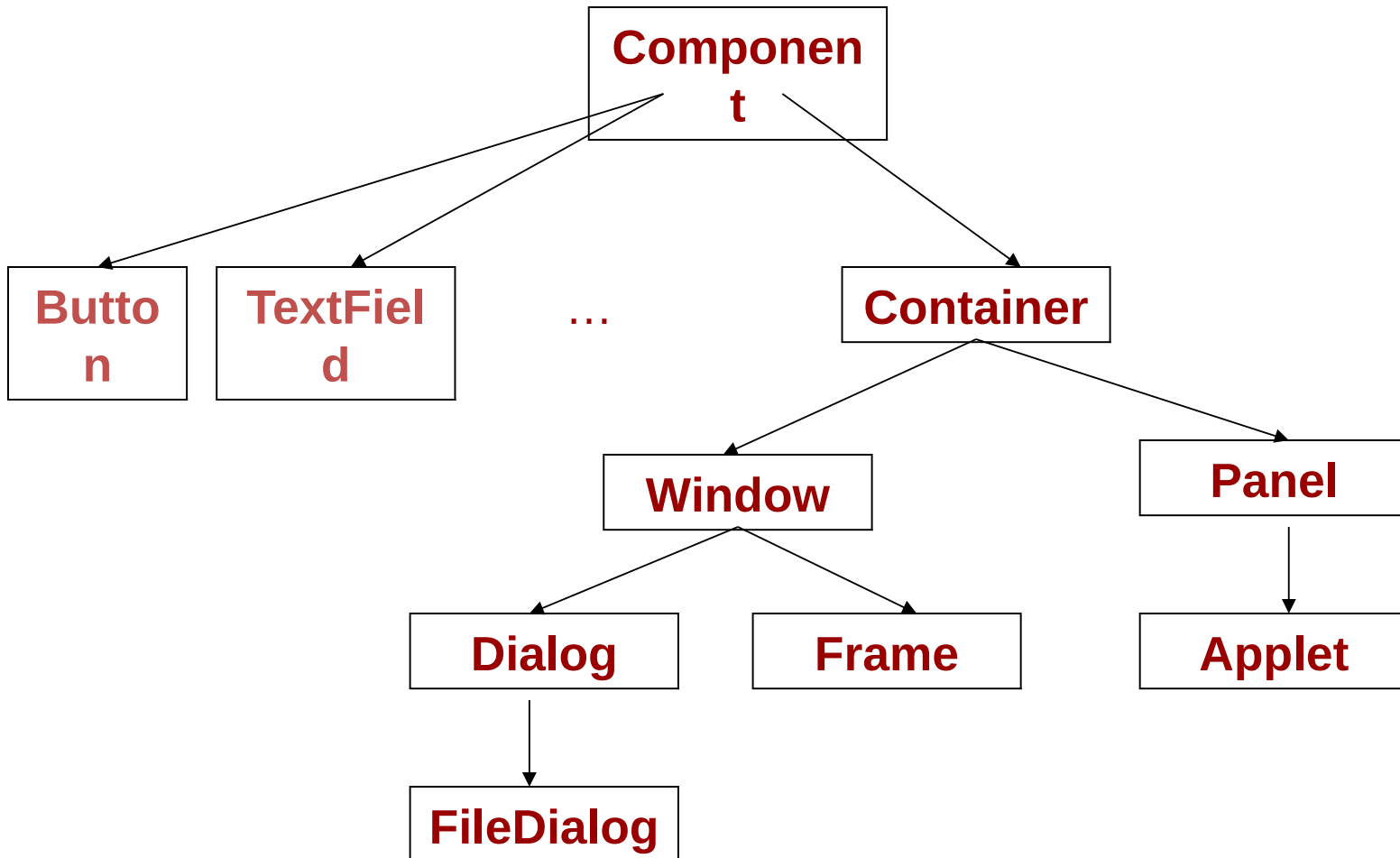
- AWT (Abstract Window Toolkit)
 - ✓ permet de créer des fenêtres (Frame), d'utiliser des boutons de commandes (Button), des zones de texte (TextField), des zones de liste, de dessiner, etc.
 - ✓ l'apparence de certains composants dépend de la plateforme d'exécution (utilise des objets de l'env. d'exécution).
- Swing propose des composants
 - ✓ indépendants de la plate-forme (écrits en Java)
 - ✓ analogues à ceux de AWT (JFrame, JButton, JTextField, etc.)
 - ✓ avancés : JTabbedPane (onglets), JTree (arbres), etc.

Ma première application avec GUI

```
import java.awt.*;  
  
public class SimpleGUI  
{  
    public static void main(String args[]){  
        Frame cadre = new Frame("Ma premiere  
fenetre");  
        cadre.setSize(300,200);  
        cadre.setVisible(true);  
    }  
}
```



Hiérarchie des composants AWT



Interfaces graphiques : Composants

- classe **java.awt.Component**
- Comportement d'un objet **Component** :
 - ✓ se dessiner (**paint**)
 - ✓ définir / obtenir taille et position du composant (**setSize**, **setMinimumSize**, etc.)
 - ✓ définir / obtenir propriétés graphiques (**setBackground**, **setForeground**, **setFont**, etc.)
 - ✓ être source d'évènements

Interfaces graphiques : Conteneurs

- **java.awt.Container** : espaces graphiques destinés à recevoir plusieurs composants (boutons, zones de texte, autres conteneurs ...) : **Panel, Dialog, Frame, Applet** ...
- La **mise en page** d'un conteneur est confiée à un gestionnaire qui se charge du placement des composants ajoutés par la **méthode add()**.
- Les panneaux (**Panel**) doivent être placés dans un espace existant (autre conteneur, navigateur Web).

Interfaces graphiques : Fenêtres

- **java.awt.Window** : sous-classe de **Container**
- Possibilité d'être visible sans nécessiter d'être incluse dans un autre composant.
- Les classes **java.awt.Dialog** (boîtes de dialogue) et **java.awt.Frame** (cadres) sont des sous-classes de **Window**
- **Cadre** : fenêtre avec bord, titre et, éventuellement menu
Généralement, une application contient un seul cadre; il est utilisé pour modifier la taille et la forme de l'interface.
- **Boîte de dialogue** : fenêtre utilisée pour afficher un message, poser une question, etc.

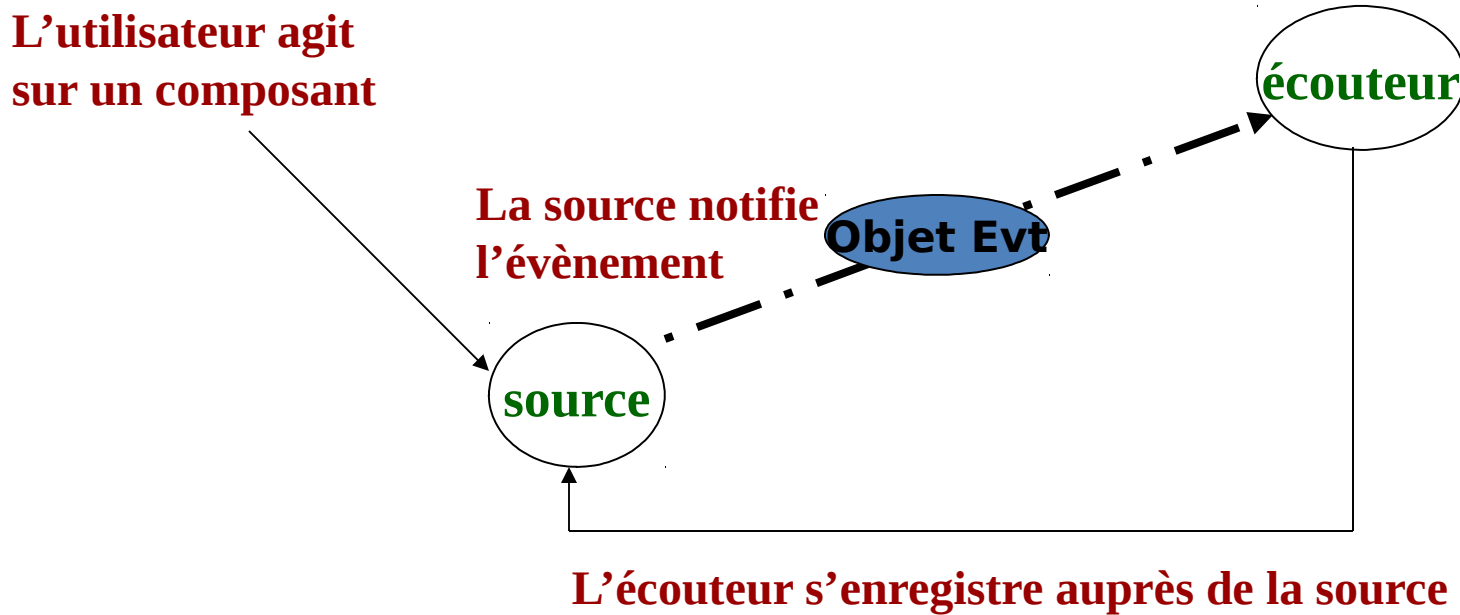
Interfaces graphiques : Exemple 1

```
import java.awt.*;
public class Fenetre extends Frame {
    public Panel p; public TextField texte;
    public Button bouton1, bouton2;
    public Fenetre() {
        super("exemple de fenetre");
        p = new Panel(); this.add(p);
        p.add( new Label("nbre a multiplier par 2") );
        p.add( texte = new TextField(20) );
        p.add( bouton1 = new Button("doubler") );
        p.add( bouton2 = new Button("fin") );
        this.pack();
    }
    public static void main(String[] args){
        Fenetre f = new Fenetre();    f.show();
    }
}
```



Interfaces graphiques : événements

- Modèle des événements de Java (1.2)



Interfaces graphiques : événements

Un événement fait interagir trois objets :

- ✓ L'objet source de l'événement (bouton cliqué)
- ✓ Un objet (classe **ActionEvent**) mémorisant l'événement (date, source, ...)
- ✓ Un écouteur d'événement (**listener**) capable de traiter l'événement (s'enregistre en implémentant l'interface correspondante)

Interfaces graphiques : types d'événements

- **ActionListener** : actions (clic sur un bouton, choix dans une liste, etc.) sur un composant.

L'interface contient une seule méthode appelée lorsqu'une action est effectuée `void actionPerformed(ActionEvent e)`

- **MouseListener** : événements souris (clics)
- **MouseMotionListener** : mouvements de la souris
- **KeyListener** : événements clavier (touche pressée/relâchée)
- **FocusListener** : acquisition ou perte focus par un composant
- **WindowListener** : actions sur une fenêtre

Interfaces graphiques : Exemple 1 (suite)

```
import java.awt.*; import java.awt.event.*;
public class Fenetre extends Frame implements ActionListener{
    public Panel p; public TextField texte; public Button ...
    public Fenetre() {
        ...
        bouton1.addActionListener(this); // la fenetre devient ecouteur
        bouton2.addActionListener(this);
    }
    public void actionPerformed(ActionEvent e){
        if (e.getSource()==bouton1){
            double x = Double.parseDouble(texte.getText());
            x = 2*x; texte.setText(""+x);
        }
        else if (e.getSource()==bouton2){
            this.dispose(); System.exit(0); // destruction fenetre et arret progr.
        }
    }
    public static void main(String[] args){ ... }
}
```

Interfaces graphiques : Exemple 2 : Détecter la souris ...

- Les événements souris sont notifiés par les méthodes suivantes de l'interface **MouseListener** :
 - **void mousePressed (MouseEvent e)**
 - **void mouseReleased (MouseEvent e)**
 - **void mouseClicked (MouseEvent e)**
 - **void mouseEntered (MouseEvent e)**
 - **void mouseExited (MouseEvent e)**
- Un écouteur d'événements souris “s'enregistre” par :
 - **void addMouseListener (MouseListener l)**

Interfaces graphiques : Exemple 2 (suite)

```
import java.awt.*;           import java.awt.event.*;

public class CadreSensible extends Frame implements MouseListener {
    CadreSensible(){
        addMouseListener(this);
        setBounds(100, 100, 300, 200);
        setVisible(true);
    }

    public void mouseClicked(MouseEvent e){
        System.out.println("Clic en (" + e.getX() + "," + e.getY() + ")");
    }

    public void mousePressed (MouseEvent e){}
    public void mouseReleased(MouseEvent e){}
    public void mouseEntered(MouseEvent e){}
    public void mouseExited(MouseEvent e){}

    public static void main(String args[]){
        new CadreSensible();
    }
}
```

Interfaces graphiques : Gestionnaires de positionnement

- La disposition des composants dans un conteneur est assurée par un gestionnaire de positionnement qui se charge de les placer correctement.
- Cela permet de rester indépendant de la plate-forme et d'éviter des calculs fastidieux ...
- Le gestionnaire par défaut est **FlowLayout** qui se contente de placer les composants les uns derrière les autres.
- Avec **FlowLayout** on peut utiliser un paramètre indiquant le type d'alignement à utiliser : **FlowLayout.RIGHT** pour droite, **FlowLayout.LEFT** pour gauche et **FlowLayout.CENTER** pour centré qui est l'alignement par défaut.

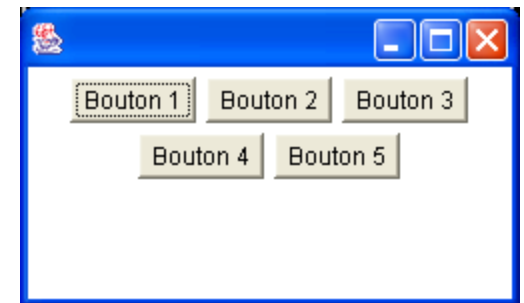
Interfaces graphiques : Gestionnaires de positionnement

- **BorderLayout** permet de placer les composants en faisant référence à une position de type géographique.
 - ✓ Pour ajouter un composant, on utilisera la méthode `add` en précisant la position géographique : **North**, **South**, **East**, **West** ou **Center**.
- **GridLayout** permet de placer les composants dans une grille formée de cases de même taille.
 - ✓ Chaque composant prend la taille d'une case.
 - ✓ Le constructeur de **GridLayout** attend 2 paramètres :
nombre de lignes et nombre de colonnes de la grille.
 - ✓ Lors de leur ajout avec la méthode `add`, les composants remplissent alors la grille ligne par ligne.

Interfaces graphiques : Gestonnaires de positionnement (exemple FlowLayout)

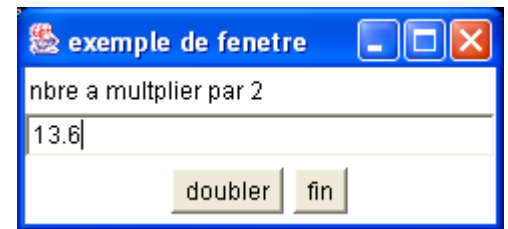
```
import java.awt.*;
public class TestFlowLayout1 extends Frame
{
    public TestFlowLayout1(){
        setLayout(new FlowLayout());
        for (int i=1; i<=5; i++)
            add(new Button("Bouton "+i));
        setBounds(100, 100, 250, 150);
        setVisible(true);
    }

    public static void main(String args[]){
        new TestFlowLayout1();
    }
}
```



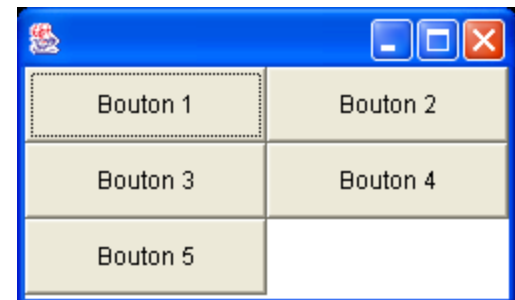
Interfaces graphiques : Gestionnaires de positionnement (exemple BorderLayout)

```
public class Fenetre extends Frame {  
    public Panel p, p_sud; public TextField texte; public Button ...  
  
    public Fenetre() {  
        super("exemple de fenetre");  
        p = new Panel(new BorderLayout()); this.add(p);  
        p.add( new Label("nbre a multiplier par 2"),BorderLayout.NORTH);  
        p.add( texte = new TextField(20) ,BorderLayout.CENTER);  
  
        p_sud=new Panel(new FlowLayout());  
        p.add(p_sud, BorderLayout.SOUTH);  
  
        p_sud.add( bouton1 = new Button("doubler") );  
        p_sud.add( bouton2 = new Button("fin") );  
        this.pack();  
    }  
    ...  
}
```



Interfaces graphiques : Gestionnaires de positionnement (exemple GridLayout)

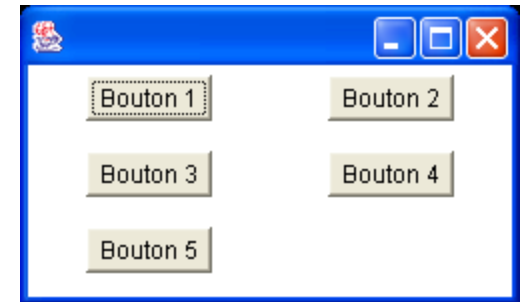
```
import java.awt.*;
public class TestGridLayout extends Frame
{
    public TestGridLayout(){
        setLayout(new GridLayout(3,2));
        for (int i=1; i<=5; i++)
            add(new Button("Bouton "+i));
        setBounds(100, 100, 250, 150);
        setVisible(true);
    }
    public static void main(String args[]){
        new TestGridLayout();
    }
}
```



Interfaces graphiques : Gestionnaires de positionnement (autre exemple GridLayout)

```
import java.awt.*;
public class TestGridLayout1 extends Frame
{
    public TestGridLayout1(){
        setLayout(new GridLayout(3,2));
        Panel p;
        for (int i=1; i<=5; i++){
            p = new Panel();
            p.add(new Button("Bouton "+i));
            add(p);
        }
        setBounds(100, 100, 250, 150);
        setVisible(true);
    }

    public static void main(String args[]){
        new TestGridLayout1();
    }
}
```



Fin du Chapitre 4.